Serverless at Lifestage

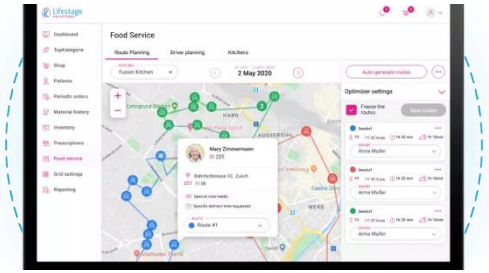
What is serverless and how we use it at Lifestage. The good parts and the challenges we faced over the past years.

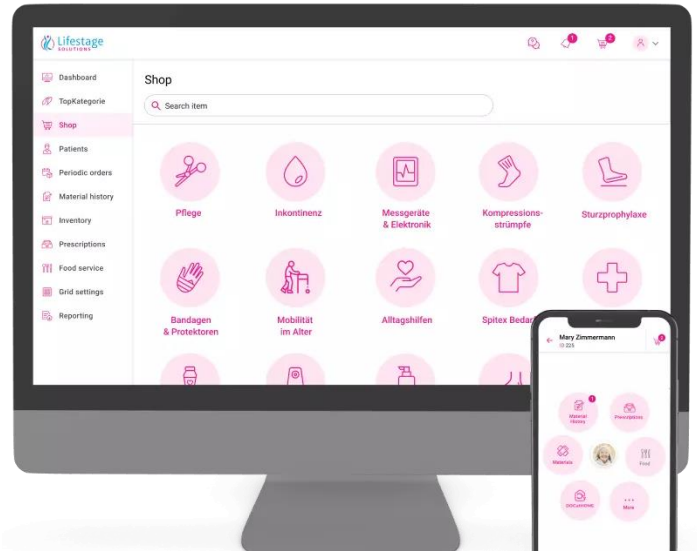
About me

- CTO at **Lifestage Solutions AG** since 2016
- Between 2000 and 2010 I've been working primarily for enterprise clients on on-premise Oracle DB and AS with Java / PL/SQL / C.
- Since 2010 mostly for startups on AWS cloud with Node.js / C++ / Lambda and DynamoDB.

Our Application

- We sell care material for the elderly to Spitex and elderly homes (APH). And we help them charging this material to insurances and patients.
- You can think about us as a small Amazon for nurses
- Among other things we handle material ordering / pick&pack / billing / automatic inventory management and warm food delivery
- We are live since 2016 and constantly adding features.
- We serve >200 organizations in 3 languages across Switzerland.
- We have ~2.5k digital scales deployed across Switzerland to track product usage



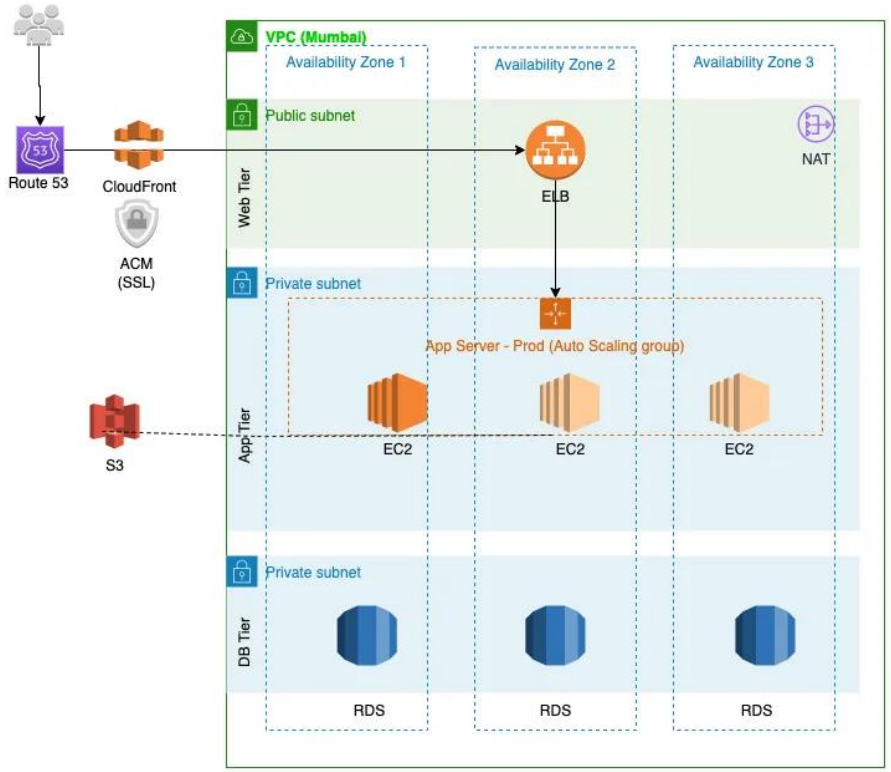


Agenda

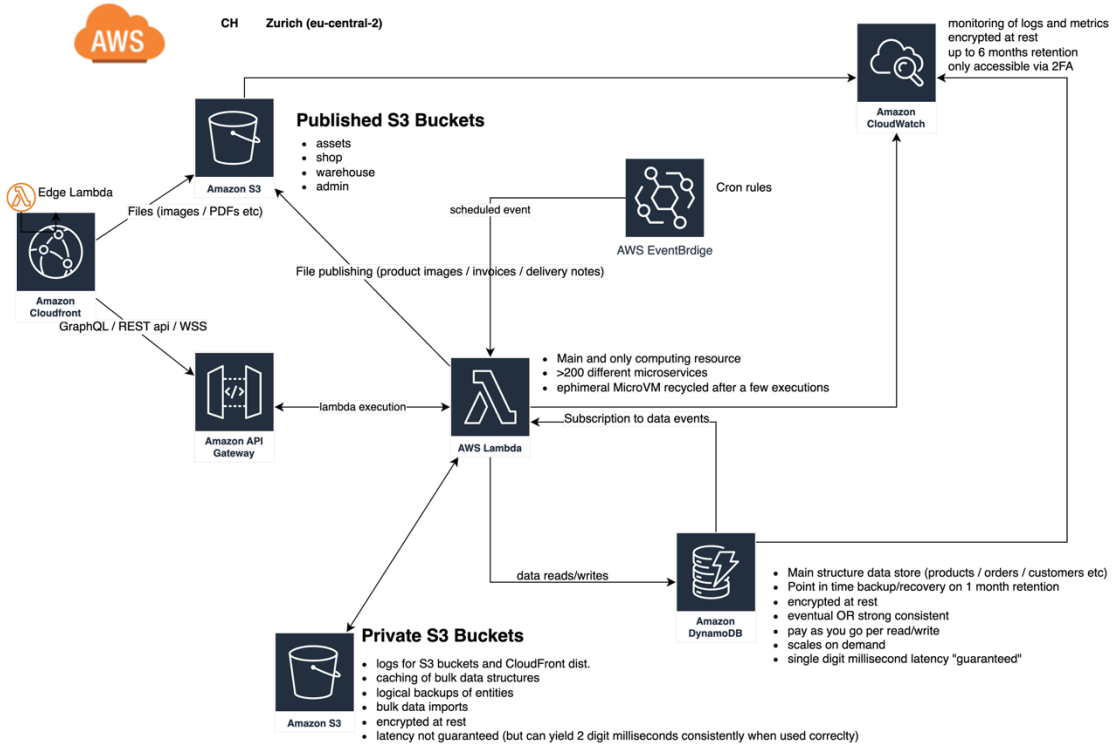
- What is “serverless”?
- Why we choose it?
- What we enjoyed AND struggled with
- An example: our application and the components we use to run it
- Q&A

“serverless”?!

Traditional 3-tier architecture (servers)



Serverless architecture (services)



What is “serverless”?

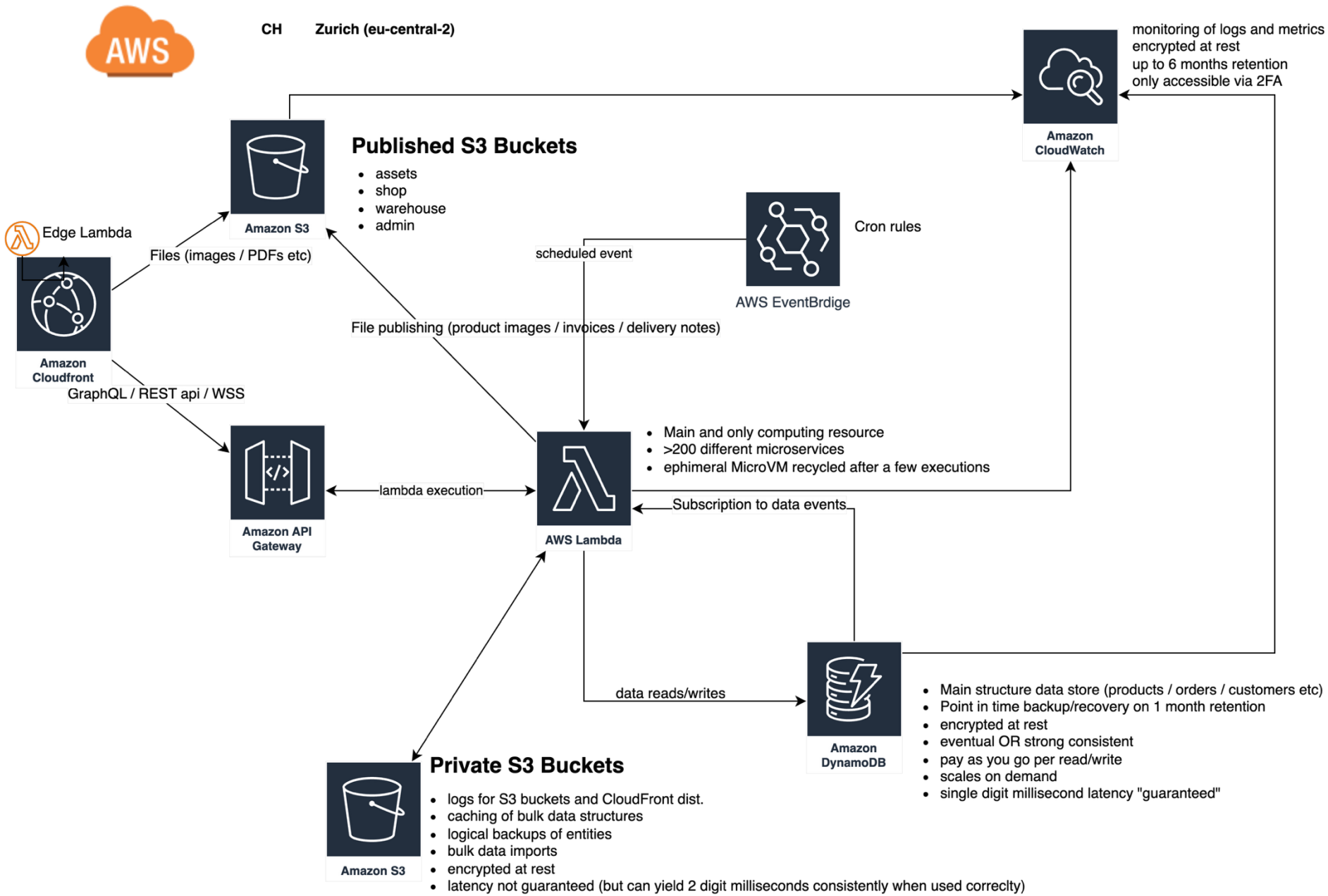
- **Managed** infrastructure for compute AND persistence (operated by your cloud vendor e.g. DDoS mitigation, OS/AS/DB updates)
- **Scalable and highly available** by design (serverless functions and persistence runs on different AZ active-active)
- **Billing model**: pay only for the resources you actually use (by execution duration or IO operations)

- **Tradeoff**: scalable simple interface *implies* some compexity is moved back into the application code (e.g. state management). No lift and shift.

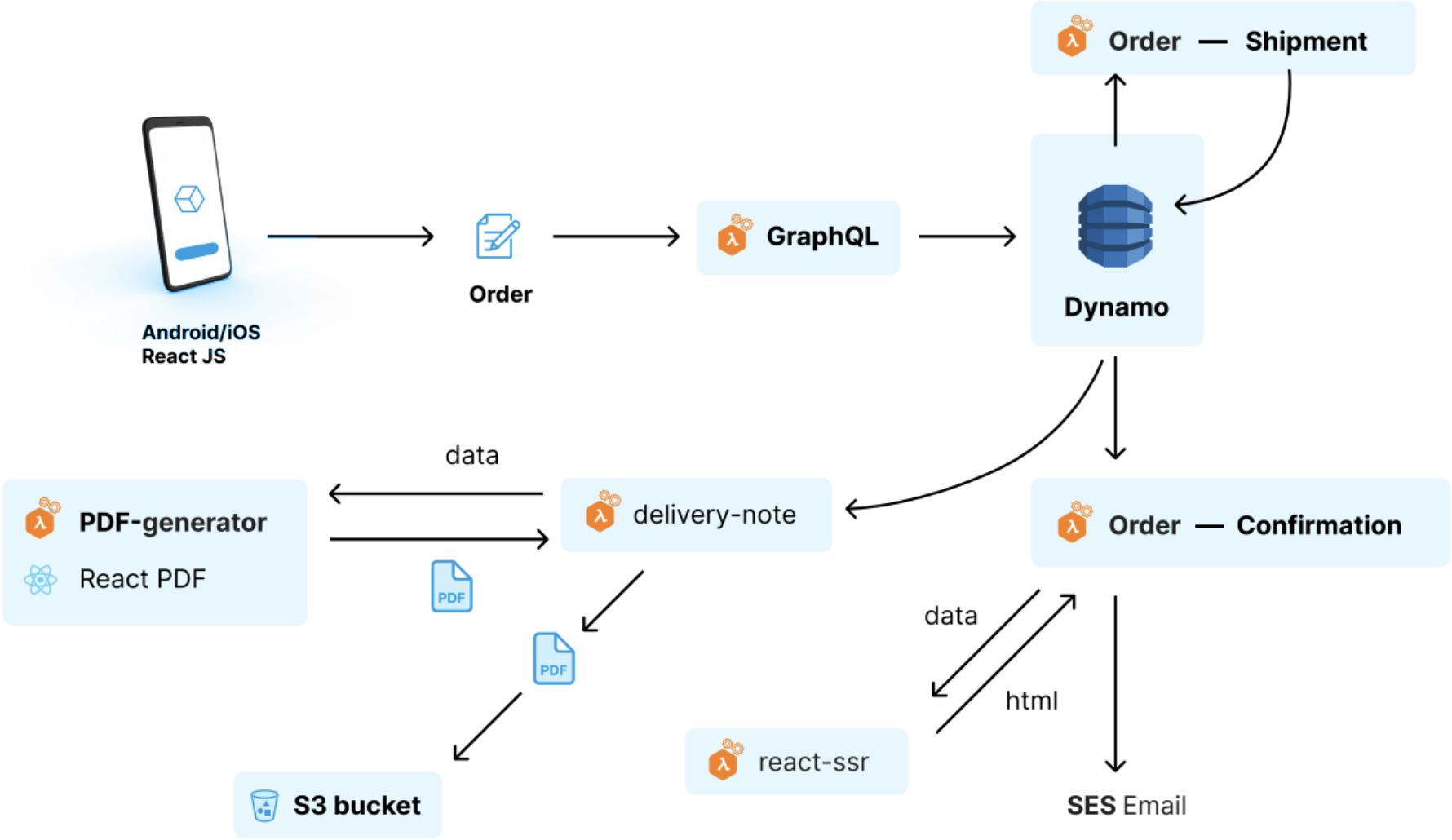
Why we choose it?

- **Move fast / break things:** focus on business logic, on customers, on everchanging regulations instead of babysitting systems
- **Billing model**: pay only for the resources you actually use / have the cost scale with revenue

Architecture components



A simplified request



The good

Resilient: if a request crashes a VM all the other requests are running on different containers / VMs, so are completely unaffected. Also memory leaks are less of a problem.

Flexible: custom runtimes depending on feature JS/C++/Python/memory size/wrap cli. Easy to add features without touching the running parts.

Low cost: when implemented efficiently if no user is active there is no running cost as compute time is billed on milliseconds of use * GB of memory used. (our ratio: revenue / 1000)

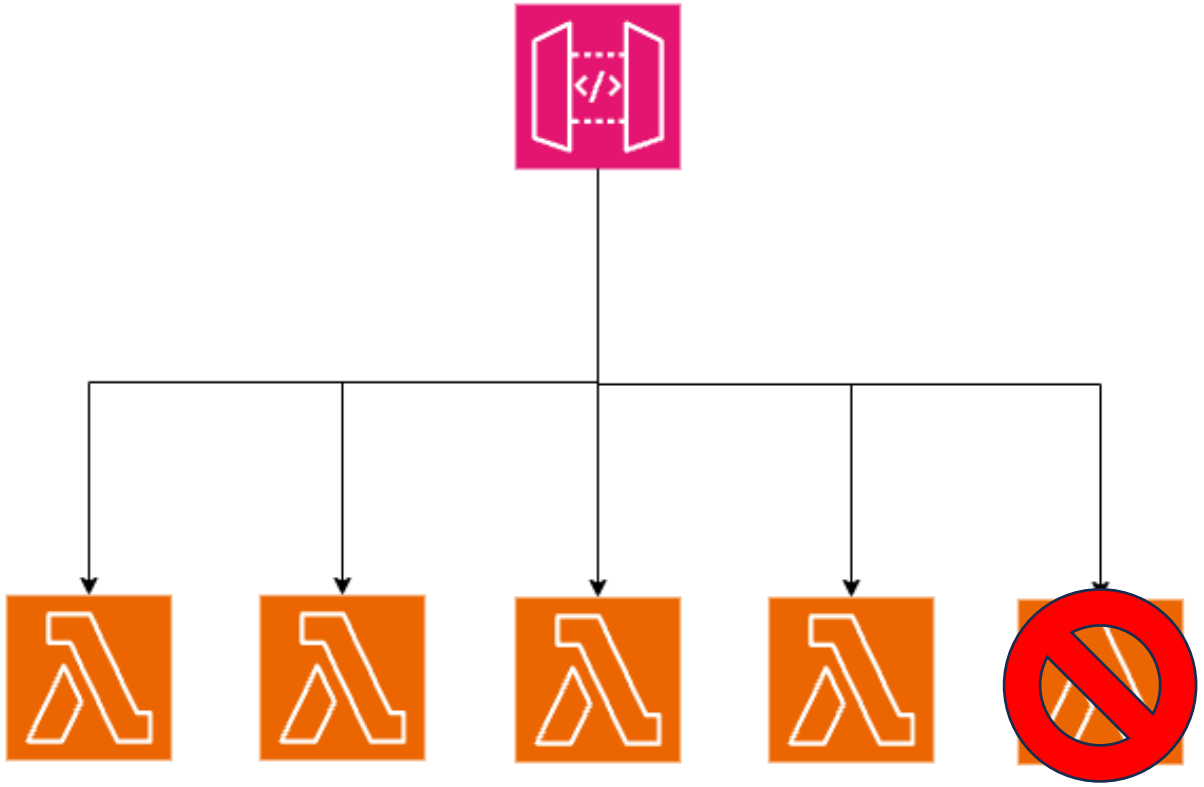
Scalable: new lambda instances are created as requests come in and kept running for a while to serve new requests

highly avaialable: (API Gateway is automatically load balancing on Lambda instances in different data centers typically 3 per region)

Low touch: No operational cost for servers. (no OS updates / AS updates / downtime / hot deployment does not affect running requests)

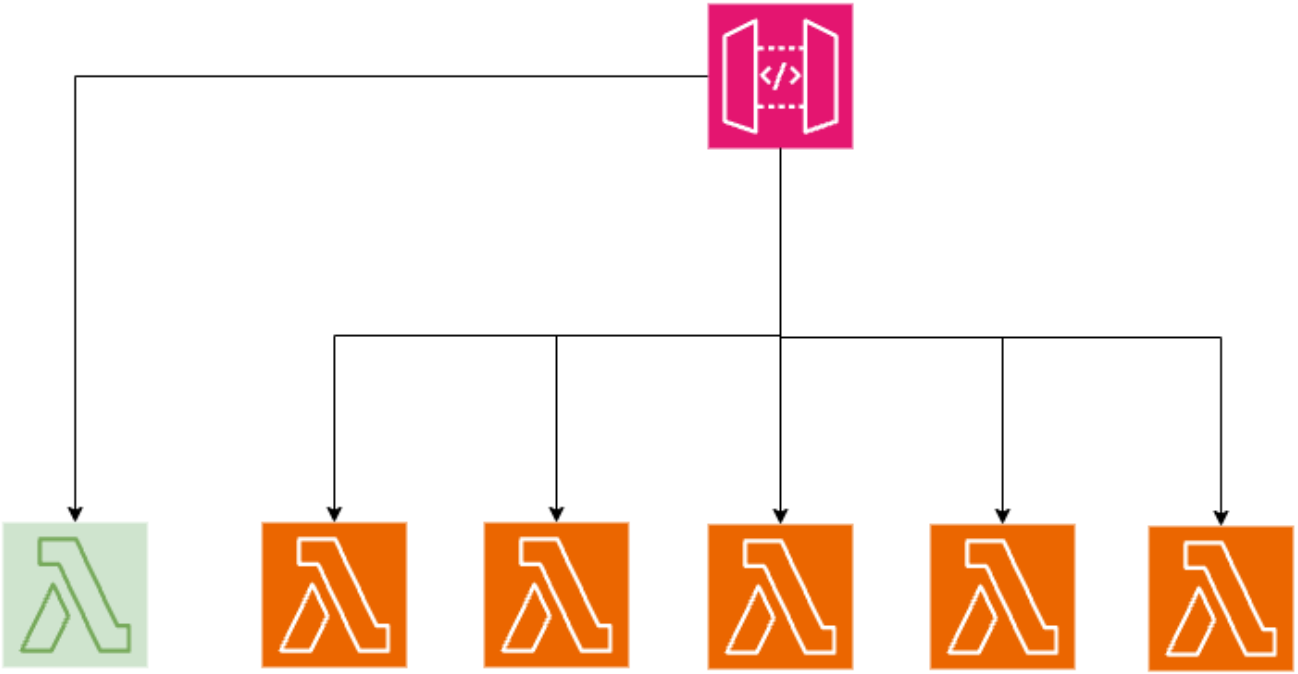
Resilient

- Each request has its own dedicated application server instance



Felixible

- Add new API endpoints without touching deployed ones
- Add new DB / Event subscribers without affecting the running ones



The “bad”

Coordination: building complicated workflows without lock-in on solutions like AWS Step Functions is tricky. Also being aware of vendor lock-in while architecting is crucial to use ONLY the relevant and reproducible portion of the offered APIs.
https://serverlessworkflow.io/

Complexity: you are always facing a very constrained execution environment (max execution time / max memory / max input/output size / disk space / code size / stateless) and so you constantly need clever solutions to work around this limitations to enjoy the benefits.

Cold starts: lambdas are on-demand containers that need initialization, so if you need low latency, you probably have to pay for provisioned concurrency.

The “ugly”

VM updates: AWS discontinues obsolete versions of VMs (Java/Node/Python etc) so if your code relies on it, you have to upgrade it and keep it fresh. Challenging at times with native extensions (C++). Especially when you have to review the impact on 100s of microservices. Needs planning or can be worked around with custom runtimes.

Local development

- We use 1 Docker container per mocked service / app, recreating the same service topology as in AWS
- We mostly use AWS services (or portions of the API) that we can easily mock locally.
- Although there are several offerings for local development replacements, we end up writing our own.

Operations via REPL

- Data import / export / plots
- Scripts
- Log investigation
- Manual Lambda execution
- Authorized granularly via IAM

Q & A

What about security?

- Shared responsibility model (infrastructure is kept secure and up to date from the cloud provider, application developers need to be sure to implement their code according to best practices e.g. OWASP)
- Lambdas are stateless and ephimeral created on invocation and get often recycled. Based on Firecracker MicroVMs for isolation (KVM). Strict runtime constraints (memory / cpu / disk space / execution roles).
- Data at rest and in transit is encrypted by default in most AWS services, and can be configured to use client owned keys or service keys.
- CloudFront has built-in DDoS mitigation and API Gateway can be granularly configured to throttle API calls blacklist IPs and validate request content before the request is passed to any compute layer.
- Custom request authorizers (session token validation / signed requests validators) are configurable both at the Edge (CDN nodes) and on API Gateway.
- You can run some Lambdas in restricted VPCs with access to crafted network resources and e.g. no internet access at all.

Examples of serverless offerings

Compute: AWS Lambda, Google Cloud Functions, MS Azure Functions

Database: AWS DynamoDB, Google Cloud Spanner, MS Cosmos DB

File storage: AWS S3, Google Cloud Storage, MZ Azure BLOB storage

Caching: MomentoHQ

Examples of NON-serverless offerings

Compute: AWS ECS, AWS EC2, AWS Elastic Beanstalk, Google AppEngine, Azure App Service

Database: AWS RDS , Azure for MYSQL/Postgre/..

File storage: AWS EFS

Caching: AWS Elasticache (Redis), Google Memorystore (Redis), Azure Cache (Redis) , AWS DAX

Example cost structure (Lambda x86 vs ARM)

Memory (MB)	Price per 1ms
128	$0.0000000023
512	$0.0000000092
1024	$0.0000000183
1536	$0.0000000275
2048	$0.0000000367
3072	$0.0000000550
4096	$0.0000000733
5120	$0.0000000917
6144	$0.0000001100
7168	$0.0000001283
8192	$0.0000001467
9216	$0.0000001650
10240	$0.0000001833

Memory (MB)	Price per 1ms
128	$0.0000000018
512	$0.0000000073
1024	$0.0000000147
1536	$0.0000000220
2048	$0.0000000293
3072	$0.0000000440
4096	$0.0000000587
5120	$0.0000000733
6144	$0.0000000880
7168	$0.0000001027
8192	$0.0000001173
9216	$0.0000001320
10240	$0.0000001467

Cost grows with memory but so does VCPUs (1-6) so if your code can use parallelism it will run faster AND cheaper on bigger Lambdas. Test test test.

Example cost structure (Dynamo)

On-Demand Throughput Type	Price
Write Request Units (WRU)	$1.67 per million write request units
Read Request Units (RRU)	$0.336 per million read request units

RRU: 1 item read up to 4KB (2 if eventual consistent)
WRU: 1 item write up to 1KB

This is the case for each table AND index, although you can limit the size of each index entry by projecting only
A few attributes from the original item and save on both index reads and writes.